

最高の AI Agent チームを 目指して

個人 OSS 「kobaamd」を 21 日間 AI 任せで作った、
自律チームの育て方の 3 フェーズ

なぜ kobaamd を作るのか

個人 OSS の動機 - 道具を作りたいかった理由 / AI 駆動を試したかった理由

① Markdown Editor が欲しかった

AI 時代、Markdown は事実上の標準フォーマット

でも既存エディタは...

- 重い (IDE 系)
- クラウド前提 (Notion / Obsidian Sync)
- Mac native 体験が薄い

→ 自分が一番使いたい道具がないなら作る

② AI 駆動を本気で試したかった

EM 歴は長い、モダン Web ・ Mac native は素人

「エンジニアじゃない自分でも、
AI に頼れば作れるのか？」

実験場としてちょうどよい OSS 規模

→ AI Agent をチームメンバーとして迎え入れる

この LT のゴール：個人 OSS で得た学びを、皆さんの業務に持ち込めるように

題材：kobaamd の規模感

2026-04-19 開始 → 21 日間で

21

プロジェクト期間

2026-04-19 → 05-10

167

累計コミット数

1 日あたり 約8 commits

≈ 100

マージ PR 数

最大 PR #110

kobaamd の UI (後で実スクショに差し替え)

Swift / SwiftUI / AppKit / macOS 14+ — Phase 3 まで完了

Phase 1 : まず AI と一緒に作ってみる

Vibe coding 期 - 触らないと限界は見えない / 3 つの AI を役割分担して走らせる



Gemini

特徴

- ・ マルチモーダル
- ・ 大量入力に強い
- ・ 検索ベースの最新情報

→ だから この役割

リサーチ・調査



Codex CLI

特徴

- ・ コーディング特化
- ・ 単価が安い
- ・ ChatGPT Plus 認証で動く

→ だから この役割

実装の主力



Claude

特徴

- ・ 設計判断・推論力が高い
- ・ コードレビューが強い
- ・ 単価は高め

→ だから この役割

オーケストレーター

発見①：能力の高い AI ほど、放っておくと抱え込む

Phase 1 で見えたこと

「あれ？最近 Codex に頼んでる雰囲気がないな...」

「Claude さん、実装どうしてる？」

→ 「私が書きました！」

観察：

能力の高いメンバーは、放っておくと
「自分でやった方が早い」と判断する。それは AI も同じ。

→ AI の能力に応じた 役割の固定化 が必要

対処：役割を CLAUDE.md に明文化する

Phase 1— コードと同じくレビュー可能な場所にルールを置く

CLAUDE.md (抜粋)

役割分担 (厳守)

- Claude はコードを書かない
- 実装は Codex CLI に依頼する
- 調査・ドキュメントは Gemini に依頼

判断基準

.swift を編集 → Codex に依頼
技術調査・選定 → Gemini に依頼
設計・レビュー → Claude が担当

効果

- ・ Claude が抱え込まなくなった
- ・ モデル単価の最適化が効く
- ・ 新しい session でもルールが効く
(再現性が出る)
- ・ ルール変更がコードレビュー対象に
(PR で議論できる)

→ 役割が固定されると、AI は本来の強みを発揮する

発見②：対話型では、人間が必ず間に入る

Phase 1 で見えたことー あるあるな1日

1 Claude 「次どうしますか？」で作業が中断

2 PR が3つ溜まってる、レビューが追いつかない

3 通知でチラ見 → 5分でも作業に戻れない

4 「あと一周だけ」で午前2時まで止まらない

5 テスト失敗の原因切り分けは結局自分

6 曖昧質問への待機状態が積み重なる

Cursor / Buddy / Claude Code、すべて『対話型』。

→ 仕組みで置き換える必要が見えた

(Phase 2 へ)

Phase 2 : 仕組みで AI チームを伸ばす

Phase 2

○ ● ○

自律化期 - Linear + .claude/agents/ + .claude/commands/ + launchd (OS の定期実行)

定常運用の規模感

13

SubAgent

.claude/agents/

28

Slash Command

.claude/commands/

3

定期実行バンドル

30 分 / 8:00 / 月 9:00

稼働実績 (2026-04-30 にパイプライン基盤を投入してから)

197

ステータス遷移回数

560

Linear API 書込回数

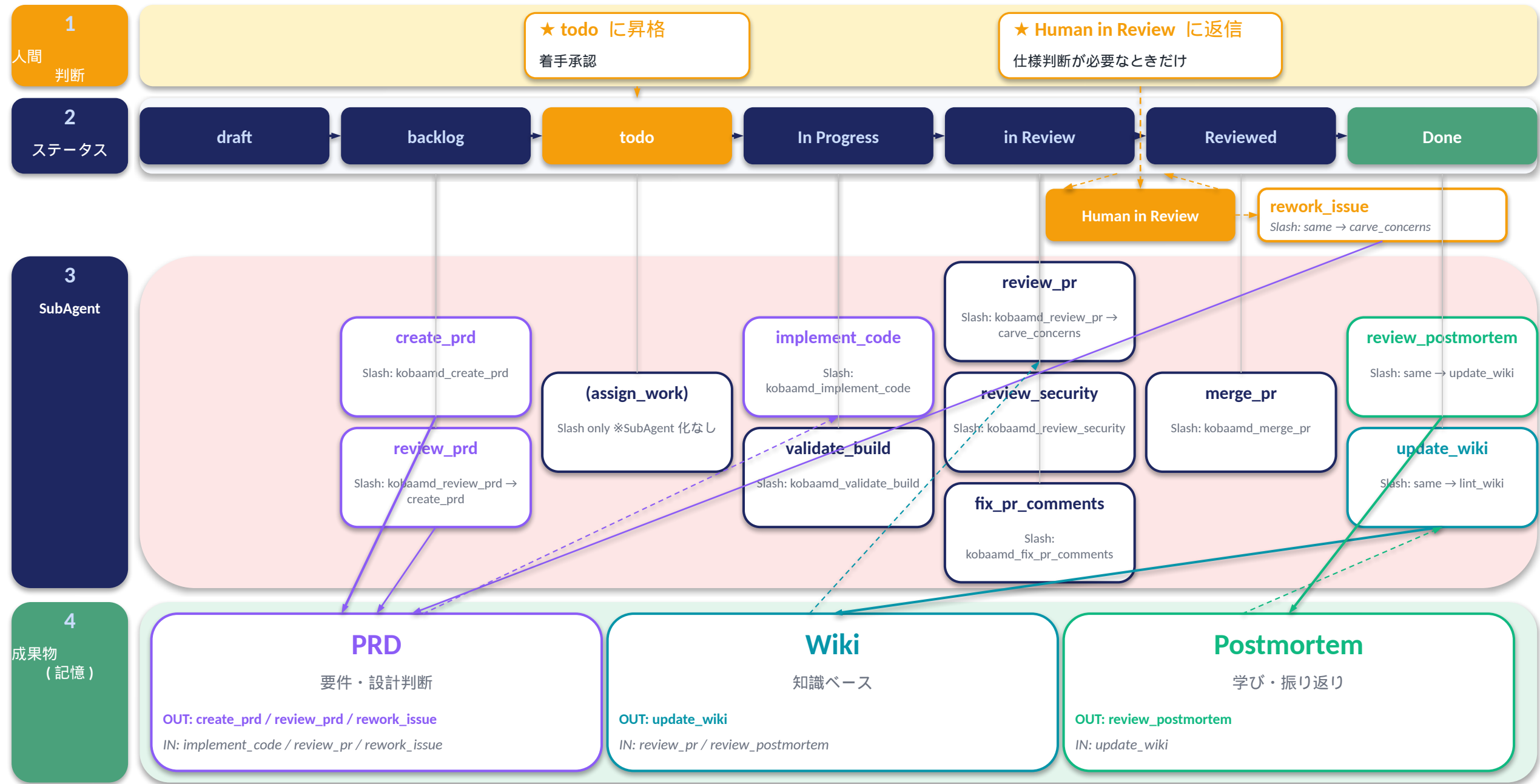
31+

Done 済 issue

人間がやるのは 2 つだけ : ① やりたいことを投げる ② 優先順位をつける

パイプラインの全体像：通常は Human in Review を通らずにマージまで進む

Phase 2



育成期：詰まりは、仕組みを育てる材料

Phase 2 — 詰まりごとに、仕組みを継ぎ足してチューニング

状態ループ

KMD-145 – In Progress ↔ Todo を 5 回以上往復
AI レビューが REQUEST_CHANGES → 修正 → 同じ指摘

コメント原文（一部）：

「PRD Section 8 影響範囲マップに記載されていない...
Sparkle 変更だけが残存」

コメントスルー

KMD-119 – Human in Review で 72 時間以上停滞
AI が新規コメントを検出できず再開しない

→ 人間がリマインドしないと進まない

トークン枯渇で停止

想定外の途中停止（rate limit / context limit）
復旧には人間の介入が必要

→ 入口でリソース見積もりが必要だった

デグレに気付かない

ビルド通れば「ヨシ！」と先に進む
UI が崩れていても AI は検知できない

→ E2E UX の自動検証は別の仕組みが必要

→ 詰まりごとに、仕組みを育てる材料が見つかる

課題 → 対応：仕組みを育てる 3 軸

Phase 2 — 仕組みは育てるもの。一度作って終わりではない

01 課題

SubAgent が起動するたびに
プロジェクト文脈を持っていない
毎回ゼロから状況把握

対応

LLM-Wiki を整備

Prompt Caching 経由で
SubAgent に即座に文脈共有

02 課題

token がすぐ尽きる
重いタスクほど枯渇しやすい

対応

Caching ・ 参照削減 ・ 軽量モデル併用

Sonnet / Haiku を併用、
推論力に応じて使い分ける

03 課題

同じ詰まりを別 issue で繰り返す
学びが個別 issue に閉じている

対応

Postmortem を Wiki へ ingest

learnings を update_wiki が
統合 → 次セッションが学習

★ 限界：E2E の UX 評価は AI にできない

「動く」と「使える」は違う。最後は人間が触らないと判断できない領域がある — Phase 3 へ

本質的発見：AI を伸ばすのは、人間の任せ方

詰まりの根本は、AI ではなく自分の任せ方にあった

01

指示が曖昧だと
出力も曖昧

「いい感じに」では
人間でも同じ結果。
期待値とゴールを言語化

02

手順を渡さないと
逐次質問される

新人が毎回上長に
確認するのと同じ。
手順は外在化する

03

AI には『積み重ね
の記憶』がない

経験で次手を推測できない。
記憶を毎回プロンプトで
再現する

新人エンジニアと同じ。期待値を伝え、手順を渡し、引き継ぎ書を残す。

対処：仕事の手順を **Slash Command / SubAgent**（つまり **md**）に落とした

－ 再現可能な「育て方」に変えた－

Phase 3 : QCD を磨く

最適化期 – 走らせながら、品質・コスト・納期を磨く

Q

Quality (品質)

E2E の UX 検証を
Gemini あたりに任せる

「動く」と「使える」のギャップを、
マルチモーダル AI で自動化して埋める

C

Cost (コスト)

軽量モデル併用で
token 消費を最適化

Sonnet / Haiku を適材適所で使い、
単価を下げながらスループットを上げる

D

Delivery (納期)

「人間が判断すべきか？」
を疑い続ける

残った人間ゲートをさらに減らせないか。
リードタイムを短くする

– 仕組みを作るのは、人間の仕事 –

伝えたい3つのこと

01

AI を道具と思わず
人だと仮定して
仕組みを作ろう

プロセス・ルール・手順の外在化が前提。
対話だけで動かそうとしない

02

AI が伸び悩んだら
仕組みで支えよう

「AI が詰まった」で終わらせない。
指示・手順・環境のどこを磨くか考える

03

急に振りすぎない
優先順位をつけて
渡そう

並列で振りすぎるとコンフリクト多発。
WIP を絞り、順番を決める

結局、AI Agent も、人間のチームと同じように育てていく

AI マネジメントは、 より良い任せ方を 磨いていく旅

Phase 1 → まず AI と一緒に作ってみる

Phase 2 → 仕組みで AI チームを伸ばす

Phase 3 → QCD を磨く - これからも仕組みを育てていく

github.com/kobaaam/kobaamd / Slack: @kobaan